Power MOSFET Aging-Test Data Analysis and Machine Learning–Based Current Forecasting

Speaker : Kuang-Chi Chen

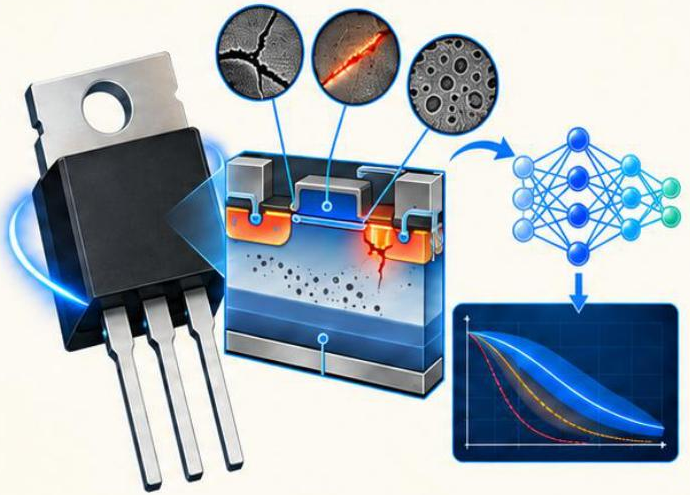


Table of Contents

Section	Page
Motivation and Objectives	1
Power MOSFET : Device and Measured Signals	2
Dataset Overview	3
Preliminary Trend and Event Analysis	5
Technical Architecture : Current Forecasting Pipeline	7
Current Forecasting : Inputs, Workflow and Models	8
Validation Strategy and Performance Evaluation	12
Discussion	18
Conclusion and Future Outlook	19

Motivation and Objectives

Motivation

Interpreting electrical measurements during accelerated aging requires careful consideration of data quality and operating conditions. A change in measured current may reflect control settings, temperature, measurement anomalies, or possible degradation. Establishing which interpretations the data support is therefore an essential step toward reliable prediction.

This study examines NASA's MOSFET thermal-overstress aging dataset to investigate measurement trends and assess whether machine learning can improve short-horizon current forecasts over simple baseline methods. The analysis emphasizes traceable data preparation, historical feature construction, and evaluation on held-out Test-number groups.

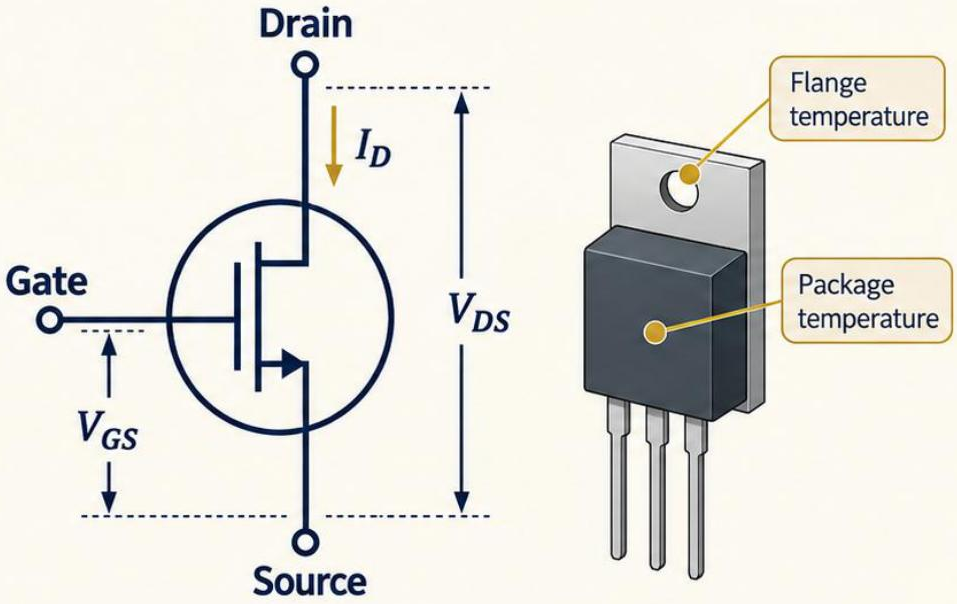
The longer-term goal is to support remaining useful life (RUL) prediction. The present study focuses on current forecasting while identifying the experimental evidence needed to establish physical-device histories and failure endpoints.

Objectives

1. **Understand the experimental data**
   Inventory the measurement files, recorded signals, controller settings, and timestamps. Identify unresolved metadata affecting interpretation.
2. **Assess data quality and construct forecasting examples**
   Screen invalid records and recording gaps, identify setting changes, and construct traceable examples within eligible recording segments.
3. **Analyze electrical trends and events**
   Examine current changes alongside available operating information, while distinguishing observed events from confirmed degradation or failure.
4. **Compare baseline and machine-learning forecasts**
   Predict future one-minute mean current using historical measurements and controller settings. Compare naïve and linear-trend baselines with Ridge regression, Random Forest, and Histogram Gradient Boosting.
5. **Evaluate performance and prediction uncertainty**
   Apply nested grouped validation, report overall and group-level errors, and assess the empirical coverage of quantile-based prediction intervals.
6. **Establish requirements for future RUL modeling**
   Identify the device mapping, measurement calibration, failure criteria, and independent validation needed before extending the analysis to lifetime prediction.

Power MOSFET: Device and Measured Signals

- **Gate–source voltage, V_{GS}:**
  Voltage between Gate and Source.
- **Drain–source voltage, V_{DS}:**
  Voltage between Drain and Source.
- **Drain current, I_D:**
  Current through the drain terminal.
- **Package and flange temperature channels:**
  Recorded thermal measurements; their interpretation depends on verified units and calibration.



In this project, an N-channel MOSFET symbol is shown to illustrate the terminals and measured electrical signals.

Interpret electrical changes alongside operating conditions and temperature before treating them as evidence of degradation.

Dataset Overview

- **Dataset:** NASA MOSFET Thermal Overstress Aging, listed as Dataset No. 13 in the NASA Prognostics Data Repository. NASA describes it as run-to-failure experiments on power MOSFETs under thermal overstress. [1]
- **Scope of this study:** 106 local MATLAB .mat files covering Test 1–42 and test_rampup. These identifiers represent file groups; their correspondence to independent physical devices requires confirmation.
- **Data structure:** Each file contains a measurement structure with controller settings, steady-state measurements and transient waveforms.
- **Recorded variables:** Voltage and current channels, package and flange temperature channels, timestamps, and operating settings such as switching frequency and duty cycle.
- **Analysis focus:** Assess data quality, compare measurements under similar operating conditions, and investigate candidate degradation trends.
- **Key limitations:** Physical units and calibration details remain partly unverified. Device identities and failure endpoints require confirmation before RUL modeling. NASA currently lists the dataset reference document as offline.
- **Identifiers:** The filenames provide Test-number and run-number identifiers. This study uses Test numbers to group related recordings. Their correspondence to unique physical MOSFETs, including continuity between runs, has not been independently verified.

[1] J. R. Celaya, A. Saxena, S. Saha, and K. Goebel, MOSFET Thermal Overstress Aging Data Set, NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA. https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository

Experimental Context and Data Interpretation

- **Measurement structure:** The "measurement" structure contains controller settings ("pwmTempControllerState"), steady-state records ("steadyState") and transient waveforms ("transient").
- **Operating conditions:** Supply settings, gate drive and duty cycle vary across some runs. Comparisons therefore require segmentation by operating condition.
- **Time interpretation:** The sum of the within-file steady-state recording spans across 106 distinct local MATLAB ".mat" files is "214.416 hours". This total includes dwell periods, low-current intervals, and any gaps within each recording span, and does not represent confirmed cumulative aging or stress exposure time.
- **Waveform interpretation:** Sample counts and sampling intervals ("dt") vary. Comparisons require a consistent physical time scale and checks of switching phase.
- **Unresolved metadata:** Device identities, cross-run relationships, units and calibration factors, and failure endpoints still require confirmation.
- **Metadata requiring verification:** The mapping between Test/run identifiers and physical MOSFET specimens, continuity between runs, measurement units and calibration factors, and failure criteria and times require confirmation.

Data Quality Findings

- **Empty waveforms and invalid timestamps:** Test 2 run 2 and Test 40 run 2 each contain one empty transient record. Records with anomalous dates require exclusion from time-based calculations.
- **Anomalous temperature-channel readings:** Test 1 contains flange-channel values near "−2,279 and −2,272", which are physically implausible "if interpreted as °C". Units and calibration require verification.
- **Constant waveforms:** Gate-source voltage waveforms are entirely constant in "33,973 of 33,980 records in Test 31" and "38,350 of 38,356 records in Test 33".
- **Acquisition changes:** Waveform sample counts and sampling intervals vary across some recordings, requiring checks before combining or comparing signals.
- **Analysis implication:** Apply quality flags and document exclusions at the affected record or channel level. These findings alone do not establish device failure.

Proposed Data Cleaning Rules

- **Preserve traceability:** Keep the original files unchanged. Record the source file, affected record or channel, and reason for each exclusion.
- **Handle invalid records:** Exclude empty waveforms from waveform analysis. Flag invalid timestamps and exclude them from duration calculations.
- **Isolate unreliable channels:** Exclude unresolved flange-temperature readings from thermal analysis and constant gate-source waveforms from gate-based feature extraction. Retain other usable channels.
- **Segment by recording conditions:** Separate intervals when operating settings, waveform sample counts, or sampling intervals change. Verify compatibility before combining segments.
- **Preserve uncertain outcomes:** Retain low-current events with quality flags. Assign failure labels only when supporting evidence confirms the failure criteria and timing.

Implementation status: A steady-state current screening and window-construction pipeline has been implemented, with file-level audit outputs. The broader waveform and channel-quality rules remain outside this forecasting implementation.

Preliminary Trend and Event Analysis

Test 38

High-current medians decrease from **2.547 to 2.378** between 0.16-0.33 h and 1.16-1.33 h, approximately **6.6%** using rounded values. Consecutive low-current records span about **3.5 hours** (1.39-4.89 h).

Test 41

High-current medians change from **2.6706 to 2.6720** between 0.20-0.53 h and 6.87-7.20 h, approximately **+0.05%**. The longest consecutive low-current sequence spans **18.34 seconds**.

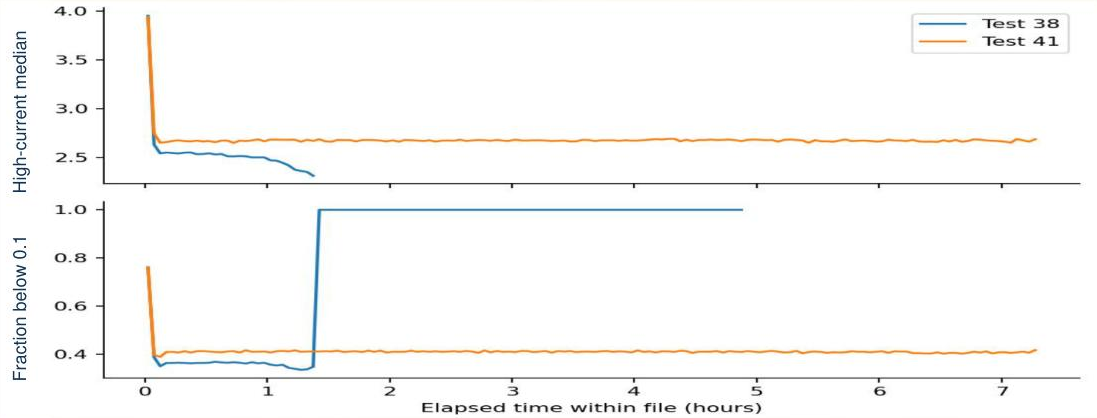


Figure 1. Summaries in 3-minute windows. High current means recorded current > 1. Windows without high-current readings remain blank. Curves summarize measurements, not model predictions.

Units: All current values and thresholds use the recorded scale. Physical units and calibration remain unverified.

Low-current fraction = $\frac{\text{the length of current reading interval below 0.1}}{\text{the length of current reading interval for all valid currents}}$ in the same window.

Link to forecasting: These high-current medians describe exploratory trends. The model predicts the mean of valid readings in a future one-minute interval, including low-current states. Thresholds do not confirm on-state operation or failure.

Physical Interpretation and RUL: Remaining Gaps

Device histories: Verify physical-device identities, links between runs, and whether recording gaps represent downtime or stress exposure.

Measurement validity: Confirm units, scaling, calibration and synchronization before calculating physical indicators.

Comparable indicators: Verify on-state intervals and operating/temperature effects before interpreting changes as degradation.

Endpoints: Confirm failure criteria and times for RUL. Define threshold-event duration and recovery rules before event modeling.

Implemented Forecasting Dataset and Target

Examples and inputs: Each window stays within one file and one recorded-setting segment. Five historical one-minute bins supply current lags, slope, variability, low-current fractions and sample counts.

Operating context: Inputs include seven recorded controller settings, including temperature setpoints. Measured package/flange temperatures are not current model features; their use requires further validation.

Target: Predict mean valid current in the one-minute interval ending five minutes after issuance, including low-current states. Example: history 10:00-10:05; issuance 10:05; target 10:09-10:10.

Eligibility and validation: Require consecutive valid bins and unchanged settings through the target. Future records determine retrospective eligibility. Test-number groups remain separate across outer folds; device independence is unverified.

File screening: 106 inventory files → 104 candidates → 83 contributing files

Exclude test_rampup.mat and Test_13_run_1 1 (error setting up this run).mat from the candidate set. Of 104 candidates, 83 yield eligible windows: **9,304 examples across 33 Test-number groups**.

Implemented output: A traceable feature-target table records historical inputs, future targets, source files, timestamps and Test-number groups. Calibrated on-state resistance and RUL remain future work.

Technical Architecture: Current Forecasting Pipeline

Implemented workflow for retrospective short-horizon current forecasting

1 Source measurements

104 candidate .mat files
Steady-state current, timestamps and seven controller settings

2 Data preparation

SciPy loading and record screening
Consecutive valid one-minute bins
Settings unchanged through target

3 Forecasting examples

Five-minute historical features
Target: one-minute mean current ending five minutes after issuance

Selection uses future setting records. Online applicability and performance across setting changes remain unverified.

9,304 eligible examples from 83 files across 33 Test-number groups

4 Nested validation grouped by Test number

OUTER 5-FOLD SPLIT

Training groups

Held-out test groups

TRAINING GROUPS ONLY

Inner 3-fold grouped tuning
Train-fitted preprocessing and refitting

ML: Ridge, Random Forest, Histogram Gradient Boosting
Intervals: quantile boosting

Test groups bypass preprocessing fitting and parameter tuning

HELD-OUT EVALUATION

Apply fitted models
Compare with persistence and linear-trend baselines
Score predictions and intervals

5 Outputs

Predictions, MAE / RMSE / bias, group-level errors, interval coverage and width

Scope: results use the raw recorded current scale. Test-number groups are not confirmed independent physical devices.

Validated failure mechanisms and remaining useful life (RUL) prediction remain outside the current results.

Current Forecasting: Inputs, Workflow and Models

Input

- Original .mat data, which include steady-state drain current, timestamps, and seven controller settings.
- Five minutes of historical measurements provide current lags, trend, variability, low-current fractions, and sampling counts.
- Screening 104 candidate .mat files produced 9,304 eligible forecasting examples from 83 files across 33 Test-number groups.

Workflow

1. Read measurements with SciPy and screen invalid records.
2. Aggregate readings into eligible one-minute bins.
3. Construct historical features and a future current target within unchanged controller-setting segments.
4. Compare persistence and linear-trend baselines with Ridge, Random Forest, and Histogram Gradient Boosting.
5. Tune parameters within training groups and evaluate predictions on held-out groups.

Output

- Predict the mean current during a one-minute interval ending five minutes after forecast issuance.
- Save held-out predictions and model errors.

File locations: Original measurements are stored in MOSFET_Thermal_Overstress_Aging/. The primary modeling outputs listed above are stored in mosfet_forecasting/results/. All paths are relative to the project directory.

The pipeline trains from the feature table held in memory and saves a copy as **features.csv**.

Scope: Results use raw recorded current units. Device identities remain unverified, and validated RUL prediction remains future work.

File	Role and contents
Original .mat files	Source inputs: measurements, timestamps, and controller settings
file_audit.csv	Audit output: screening information, inclusion status, and example counts
features.csv	Saved modeling table: historical predictors, future targets, timestamps, and group identifiers
predictions.csv	Prediction output: held-out predictions, observed targets, model names, and outer-fold numbers
metrics_overall.csv	Evaluation output: overall MAE, RMSE, bias, and mean MAE across groups
metrics_per_group.csv	Evaluation output: errors for each held-out Test-number group
metrics_per_file.csv	Evaluation output: errors for each contributing source file
prediction_intervals.csv	Uncertainty output: lower and upper bounds, observed targets, coverage flags, and interval widths
interval_coverage_per_group.csv	Uncertainty summary: coverage and mean interval width for each Test-number group
tuning.json	Model-selection record: selected hyperparameters and inner-validation scores
run_metadata.json	Run information: prediction horizon, history length, dataset counts, and grouping limitations

mosfet_forecasting/pipeline.py

```python
# Standard Python libraries
from pathlib import Path
import argparse, gc, hashlib, json, os, platform, re

# Numerical analysis and MATLAB data loading
import numpy as np
import pandas as pd
from scipy.io import loadmat

# Machine learning
from sklearn import __version__ as sklearn_version
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GroupKFold, GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error
from threadpoolctl import threadpool_limits
# Define controller inputs
ROOT = Path(__file__).resolve().parents[1]
SETTINGS = ['lowTemp', 'highTemp', 'shutdownTemp', 'gateVoltage', 'supplyVoltage', 'switchingFrequency', 'dutyCycle']

def records(x):
    return [x] if isinstance(x, dict) else list(np.asarray(x, dtype=object).ravel())

def dates(values):
    t = pd.DatetimeIndex(pd.to_datetime(values, format='%m/%d/%Y %H:%M:%S.%f', errors='coerce'))
    return t.where((t.year >= 2000) & (t.year <= 2100))
# Read original measurements
def extract(path):
    """One MAT at a time. Transient arrays are loaded by SciPy but not used."""
    m = loadmat(path, simplify_cells=True, variable_names=['measurement'])['measurement']
    ss = records(m['steadyState'])
    frame = pd.DataFrame([{'time': r.get('date'), 'current': r.get('timeDomain', {}).get('drainCurrent', np.nan)} for r in ss])
    frame['time'] = dates(frame.time)
    frame['current'] = pd.to_numeric(frame.current, errors='coerce')

    # Clean records
    audit = {'file': path.name, 'bytes': path.stat().st_size, 'mtime_ns': path.stat().st_mtime_ns, 'steady_records': len(frame)}
    audit['invalid_time_or_current'] = int((frame.time.isna() | ~np.isfinite(frame.current)).sum())
    frame = frame[frame.time.notna() & np.isfinite(frame.current)].copy()
    audit['backward_timestamp_steps'] = int((frame.time.diff().dt.total_seconds() < 0).sum())
    duplicate = frame.time.duplicated(keep=False)
    audit['duplicate_timestamp_records_removed'] = int(duplicate.sum())
    frame = frame[~duplicate].sort_values('time')
```

```
# Create one-minute bins
bins = []
for minute, g in frame.groupby(frame.time.dt.floor('min')):
    gaps = g.time.diff().dt.total_seconds()
    # Complete minute coverage, finite settings, stable commanded settings.
    if len(g) < 10 or (g.time.iloc[-1]-g.time.iloc[0]).total_seconds() < 45 or gaps.max() > 5:
        continue
    if g.segment.isna().any() or g.segment.nunique() != 1 or g.filter(like='setting_').isna().any().any():
        continue
    bins.append({'time': minute, 'mean': g.current.mean(), 'std': g.current.std(ddof=0),
                 'low_fraction': (g.current < .1).mean(), 'count': len(g),
                 'segment': g.segment.iloc[0], **g.filter(like='setting_').iloc[-1].to_dict()})

# Construct features and target
def features(b, filename, history, horizon):
    out = []
    if b.empty:
        return out
    b = b.sort_values('time').reset_index(drop=True)
    for i in range(history - 1, len(b) - horizon):
        span = b.iloc[i-history+1:i+horizon+1]
        if not (span.time.diff().dropna() == pd.Timedelta(minutes=1)).all() or span.segment.nunique() != 1:
            continue
        past = b.iloc[i-history+1:i+1]
        y = past['mean'].to_numpy()
        slope = np.polyfit(np.arange(history), y, 1)[0]
        row = {'file': filename, 'test_group': re.match(r'(Test_\d+)_', filename).group(1),
               'history_start': past.time.iloc[0], 'prediction_time': b.time.iloc[i]+ pd.Timedelta(minutes=1),
               'target_start': b.time.iloc[i+horizon], 'target_end': b.time.iloc[i+horizon] + pd.Timedelta(minutes=1),
               'target': b['mean'].iloc[i+horizon], 'persistence': y[-1],
               'linear_trend': y[-1] + slope*horizon,
               'x_last': y[-1], 'x_mean': y.mean(), 'x_std': y.std(), 'x_slope': slope,
               'x_min': y.min(), 'x_max': y.max(), 'x_within_std': past['std'].mean(),
               'x_low_fraction': past.low_fraction.mean(), 'x_count': past['count'].mean()}
        row.update({'x_lag_' + str(j): y[-1-j] for j in range(history)})
        row.update({'x_' + k: b[k].iloc[i] for k in b.columns if k.startswith('setting_')})
        out.append(row)
    return out
```

```
# Separate outer training/test groups
cols = [c for c in df if c.startswith('x_') and not (without_settings and
        c.startswith('x_setting_'))]
groups = df.validation_group
if groups.nunique() < 4:
    raise ValueError('At least four eligible validation groups are required for
          nested grouped validation.')
outer = GroupKFold(n_splits=min(folds, groups.nunique()))
predictions, tuning, intervals = [], [], []
for fold, (train, test) in enumerate(outer.split(df[cols], df.target, groups), 1):
    tr, te = df.iloc[train], df.iloc[test]

# Configure inner validation and models
    inner = GroupKFold(n_splits=min(3, tr.validation_group.nunique()))
    models = {
        'Ridge': (make_pipeline(StandardScaler(), Ridge()),
                  {'ridge__alpha': [1., 100.]}),
        'RandomForest': (RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=2),
                         {'min_samples_leaf': [5, 20]}),
        'HistGradientBoosting': (HistGradientBoostingRegressor(max_iter=100,
                                 early_stopping=False, random_state=42),
                                 {'max_leaf_nodes': [7, 15]})
```

```
# Tune and predict
results = {'Persistence': te.persistence.to_numpy(),
           'LinearTrend': te.linear_trend.to_numpy()}
for name, (model, grid) in models.items():
    search = GridSearchCV(
        model, grid, cv=inner,
        scoring='neg_mean_absolute_error', n_jobs=1)
    # inner creates training/validation subdivisions
    # within that pool.
    search.fit(tr[cols], tr.target,
               groups=tr.validation_group)
    # tr is the outer training pool.
    results[name] = search.predict(te[cols])
    # te contains the held-out outer test examples.
    tuning.append({'fold': fold, 'model': name,
                   'parameters': search.best_params_,
                   'inner_MAE': -search.best_score_})

# Export predictions
for name, pred in results.items():
    p = te[['file', 'test_group',
            'validation_group', 'prediction_time',
            'target_start', 'target_end',
            'target', 'persistence']].copy()
    p['model'], p['prediction'], p['fold'] = (
        name, pred, fold)
    predictions.append(p)
print(f'Completed outer fold {fold}; held out '
      f'{te.validation_group.nunique()} groups',
      flush=True)
pred = pd.concat(predictions, ignore_index=True)
pred.to_csv(out / 'predictions.csv', index=False)

# Save modeling table
df.to_csv(args.output / 'features.csv', index=False)
```

```
# Calculate and export errors
def metrics(g):
    e = g.prediction - g.target
    return {'n': len(g),
            'MAE': mean_absolute_error(
                g.target, g.prediction),
            'RMSE': np.sqrt(mean_squared_error(
                g.target, g.prediction)),
            'bias': e.mean(),
            'positive_error_fraction': (e > 0).mean()}
rows = []
for (name, group), g in pred.groupby(
        ['model', 'validation_group']):
    rows.append({'model': name,
                 'validation_group': group,
                 **metrics(g)})
per = pd.DataFrame(rows)
per.to_csv(out / 'metrics_per_group.csv',
           index=False)
pd.DataFrame([
    {'model': name, 'file': file, **metrics(g)}
    for (name, file), g in pred.groupby(
        ['model', 'file'])
]).to_csv(out / 'metrics_per_file.csv',
          index=False)
overall = pd.DataFrame([
    {'model': name, **metrics(g),
     'macro_group_MAE': per.loc[
         per.model == name, 'MAE'].mean()}
    for name, g in pred.groupby('model')
])
overall.to_csv(out / 'metrics_overall.csv',
               index=False)
(out / 'tuning.json').write_text(
    json.dumps(tuning, indent=2), encoding='utf-8')
```

Run these two commands in PowerShell:

```
Set-Location -LiteralPath 'project_directory'
& '.\.venv-mosfet\Scripts\python.exe' '.\mosfet_forecasting\pipeline.py'
```

Please replace the project_directory with the path to the current project folder.

Note

positive_error_fraction is the fraction of predictions exceeding the observed current. It is not Precision or Accuracy.

Validation Strategy and Performance Evaluation

1. Training, Validation and Testing

The study used **nested grouped cross-validation** to evaluate 9,304 forecasting examples from 83 files across 33 Test-number groups. All eligible runs sharing a Test number remained in the same group.

The **outer five-fold procedure** held out groups for performance evaluation. Within each outer training pool, **three-fold grouped cross-validation** selected model hyperparameters using mean absolute error (MAE). After parameter selection, each model was refitted on the complete outer training pool and evaluated on the held-out groups.

Ridge feature standardization was fitted within the training pipeline. The feature definitions were fixed for the primary experiment.

Outer fold	Training-pool examples	Training-pool groups	Held-out examples	Held-out groups
1	7,440	27	1,864	6
2	7,444	26	1,860	7
3	7,440	26	1,864	7
4	7,451	27	1,853	6
5	7,441	26	1,863	7

The training pool in each outer fold was further divided into inner training and validation subsets. Each example received one held-out prediction per method across the five outer folds.

2. Validation Scope and Operating Transitions

What the evaluation supports

Five outer folds hold out Test-number groups: 9,304 examples, 83 files, 33 groups.
Physical-device identities remain unverified. Cross-device generalization is unconfirmed.

Which operating intervals qualify

Five historical minutes predict a future one-minute mean ending five minutes after issuance.
Eligibility requires valid consecutive bins and unchanged recorded settings through the target.
This retrospective selection uses future setting records. Online applicability remains unverified.

What remains untested

Windows crossing setting changes are excluded. Their accuracy and interval coverage are unknown.
Unchanged commands do not guarantee unchanged actual drive, load or temperature.
Next: score stable and transition periods separately using eligibility known at prediction time.

Supported claim

Exploratory current forecasting on held-out Test-number groups during retrospectively selected intervals with unchanged recorded controller settings.

3. Evaluation Metrics

The target is a continuous current measurement. Performance was evaluated using predictions for the held-out examples.

Let:

- y_i: observed mean current in the target interval for example i.
- $\hat{y}_i$: predicted mean current for that interval.
- N: total number of evaluated examples, **9,304.**
- G: number of evaluated Test-number groups, **33.**
- $\mathcal{I}_g$: set of examples belonging to group g.
- $n_g = |\mathcal{I}_g|$: number of examples in group g.

Define the prediction error as: $e_i = \hat{y}_i - y_i$

Mean Absolute Error (MAE)

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|\hat{y}_i - y_i|$$

MAE measures the average absolute prediction error. **Lower values indicate better performance.**

Root Mean Squared Error (RMSE)

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2}$$

RMSE gives greater weight to large errors by squaring each error before averaging. **Lower values indicate better performance.**

Mean MAE Across Groups

First, calculate the MAE for each Test-number group:

$$\mathrm{MAE}_g = \frac{1}{n_g}\sum_{i\in\mathcal{I}_g}|\hat{y}_i - y_i|$$

Then average these group errors:

$$\text{Macro Group MAE} = \frac{1}{G}\sum_{g=1}^{G}\mathrm{MAE}_g$$

This metric gives **equal weight to each Test-number group**, regardless of its number of examples.

By comparison, overall MAE weights groups according to their example counts:

$$\mathrm{MAE} = \sum_{g=1}^{G}\frac{n_g}{N}\mathrm{MAE}_g$$

Bias

$$\mathrm{Bias} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)$$

- **Positive bias:** predictions are higher than observations on average.
- **Negative bias:** predictions are lower than observations on average.
- **Bias near zero:** little average directional error, although positive and negative errors may cancel.

Reporting note: These metrics are calculated separately for each method using the combined held-out predictions from the five outer folds. All four metrics retain the units of the target, reported here as **raw recorded current units**. Test-number groups are not yet confirmed independent physical devices.

Metric	Interpretation
MAE	Average absolute difference between predicted and observed current
RMSE	Error measure that gives greater weight to large prediction errors
Mean MAE across groups	Average of the individual held-out groups' MAEs, giving each group equal weight
Bias	Average predicted minus observed current; positive values indicate overprediction

4. Model Comparison

Simple forecasting methods can serve as benchmarks for evaluating more complex models.[2] The naïve forecasting method uses the latest observed value as the forecast for future observations.[2] This study applies that rule to the latest observed one-minute mean current and labels the method Persistence in mosfet_forecasting/pipeline.py.

For the linear-trend baseline implemented in this study, a least-squares slope is estimated from the previous five one-minute means. The forecast equals the latest observed mean plus the estimated slope, expressed per minute, multiplied by the five-minute forecast horizon.

The following results combine the held-out predictions from all five outer folds. Lower values indicate better performance. Errors are expressed in raw recorded current units.

Method	MAE	RMSE	Mean MAE across groups
Ridge regression	**0.02050**	**0.04300**	**0.03236**
Naïve forecast (Persistence)	0.02437	0.04615	0.03524
Random Forest	0.02599	0.05164	0.04593
Histogram Gradient Boosting	0.03727	0.08191	0.04717
Linear trend	0.03935	0.06222	0.05203

Source: This study's nested grouped cross-validation results, exported to mosfet_forecasting/results/metrics_overall.csv.

Ridge regression achieved the lowest aggregate errors in this experiment. Its overall MAE and mean MAE across groups were approximately 15.9% and 8.2% lower than the naïve forecast, respectively. These reductions were calculated using the unrounded results.

The naïve forecast remained a strong baseline because its prediction errors were lower than those of Random Forest and Histogram Gradient Boosting under the evaluated configurations. Greater model complexity therefore did not necessarily improve forecasting performance in this experiment.

[2] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., OTexts, 2021, Section 5.2, 'Some simple forecasting methods.'

https://otexts.com/fpp3/toolbox.html

5. Prediction Uncertainty

I.) For each outer fold, two additional Histogram Gradient Boosting models were fitted to the training data to estimate the 5th and 95th conditional quantiles of future mean current. Their predictions defined a nominal 90% prediction interval for each held-out example.

II.) Across the five outer folds, 8,244 of 9,304 observed targets fell within their predicted intervals, giving approximately 88.6% empirical coverage, compared with the nominal level of 90%. Coverage varied across Test-number groups, and some groups contained few examples. These results therefore provide preliminary uncertainty estimates without guaranteed 90% coverage.

For instance,

Test group	Examples	Inside interval	Coverage
Test 16	21	7	33.3%
Test 27	111	43	38.7%
Test 28	13	8	61.5%
Test 32	85	59	69.4%

III.) The intervals were generated by the quantile Gradient Boosting models. They do not describe uncertainty around Ridge predictions or quantify remaining-life overestimation risk, because the prediction target is future mean current.

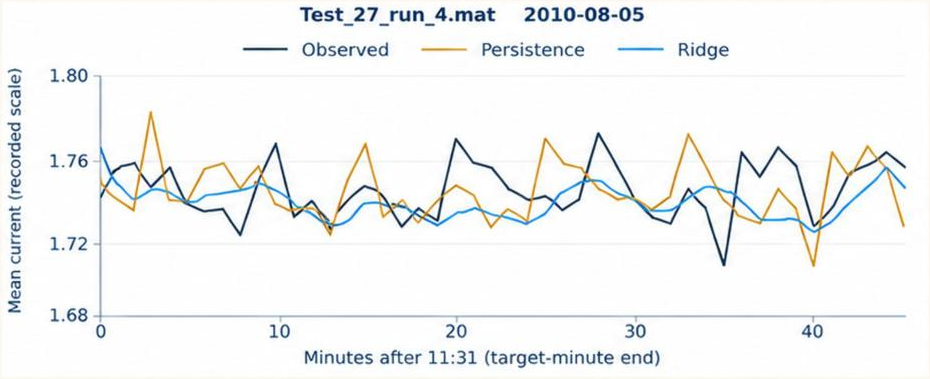
Source: This study's held-out interval predictions in mosfet_forecasting/results/prediction_intervals.csv, with group-level summaries in mosfet_forecasting/results/interval_coverage_per_group.csv.

6. How Forecasts Support Engineering Judgement

Post-hoc cases: lowest and highest Ridge MAE among Test groups with at least 100 examples

Lower error: Test 27 (111 examples)

Group MAE: Ridge 0.01270 / Persistence 0.01683



Test_27_run_4.mat 2010-08-05

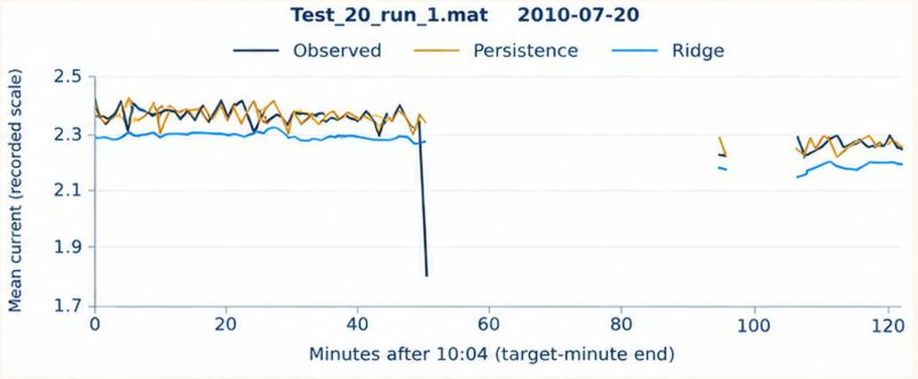
Fixed settings: duty 40, supply 5, gate 10

Temperature setpoints for the evaluated segments: 229/230

Largest run error: +0.03591 at 12:05–12:06. Ridge tracks this run closely. This does not confirm device health.

Higher error: Test 20 (132 examples)

Group MAE: Ridge 0.07221 / Persistence 0.03117



Test_20_run_1.mat 2010-07-20

Fixed settings: duty 40, supply 5, gate 10

Temperature setpoints for the evaluated segments: 209/210

Persistent group bias: −0.06126. At 10:53–10:54, observed 1.795 vs Ridge 2.292 / Persistence 2.356. Package-temperature mean falls 121.46 to 117.07.

Engineering follow-up

Investigate model bias and thermal/control-state effects. Confirm synchronized current, $\frac{V_{GS}}{R_{DS}}$, actual duty cycle and temperature calibration before attributing the change to degradation.

Raw recorded scales. Charts show representative runs, with gaps where targets were not evaluated.

Sources: This study's predictions.csv and features.csv; original measurements from Test_27_run_4.mat and Test_20_run_1.mat.

Discussion

Interpretation of Results

- **Ridge achieved the lowest aggregate errors.** Its overall MAE was 0.02050, compared with 0.02437 for the naïve forecast, a reduction of approximately 15.9%. Mean MAE across groups also decreased from 0.03524 to 0.03236, an improvement of approximately 8.2%. The smaller group-averaged improvement shows that the reported benefit depends on how groups are weighted.
- **The naïve forecast remained competitive.** Its MAE of 0.02437 was lower than Random Forest's 0.02599 and Histogram Gradient Boosting's 0.03727. Recent current values therefore provided a useful short-horizon reference for the selected intervals.
- **Greater model complexity did not guarantee improvement.** These results apply to the features, parameter candidates, and validation procedure evaluated in this study. They do not establish that tree-based methods are generally unsiutable for MOSFET forecasting.
- **Uncertainty varied substantially by Test group.** The nominal 90% intervals achieved approximately 88.6% overall coverage, but coverage was only 33.3% for Test 16 (7/21 examples) and 38.7% for Test 27 (43/111 examples). These examples show that pooled coverage can conceal poor performance in particular groups. Small sample counts and dependence between examples also limit interpretation.

Limits of Interpretation

- Test-number groups have not been confirmed as independent physical devices.
- Evaluation covers retrospectively selected intervals with unchanged controller settings.
- Overlapping examples are dependent.
- Current changes may reflect operating or measurement effects as well as degradation.

The current results do not establish failure mechanisms or RUL performance.

Conclusion and Future Outlook

Conclusions

- Evaluated **9,304 examples from 83 files across 33 Test-number groups** using nested grouped validation.
- Ridge achieved **MAE = 0.02050** and **RMSE = 0.04300** in the recorded current scale.
- Quantile Gradient Boosting intervals achieved approximately **88.6% empirical coverage** at a nominal 90% level.

Future Outlook

1. **Verify experimental metadata:** Confirm physical-device mapping, measurement units, and calibration factors.
2. **Strengthen validation:** Finalize the modeling procedure and evaluate independent data, operating transitions, and group-specific errors.
3. **Improve uncertainty assessment:** Examine both interval coverage and width, and evaluate any calibration procedure on separate data.
4. **Extend the prediction target:** Develop event or RUL models only after establishing defensible endpoints and device histories.

Final statement: This work provides an exploratory foundation for MOSFET reliability-data analysis and current forecasting. Validated remaining useful life prediction remains a future objective.